

SIMD

1 概念介绍

SIMD (Single Instruction, Multiple Data, 单指令多数据) 是一种并行计算技术, 广泛应用于现代处理器中, 用于加速对大量数据执行相同操作的场景。其核心思想是: 一条指令同时作用于多个数据元素, 从而在单位时间内完成更多计算, 提高吞吐量。

2 SIMD 和并行技术的关系

2.1 SIMD 在并行技术中的定位

表 1: 并行技术分类

类型	核心	特点	用例
指令级并行 (ILP)	单线程内多条指令同时执行	由 CPU 流水线、超标量、乱序执行等硬件机制实现	现代 CPU 自动并行执行不相关的指令
数据级并行 (DLP)	同一条指令作用于多个数据	适合规则、批量的数据操作	向量加法、图像处理
任务级并行 (TLP)	多个任务/线程同时执行	通过多核、多线程实现, 任务可不同	线程池
请求级并行 (RLP)	多个独立请求并行处理	常应用于分布式系统	多用户同时访问数据库

SIMD 属于数据级并行 (DLP) 的一种实现方式, 专注于通过单条指令同时处理多个数据元素来提升计算效率。和多线程 (TLP) 不同, SIMD 不创建新线程, 不涉及线程调度或同步开销, 而是在单个线程内提升计算吞吐量。

2.2 SIMD 与其他并行技术的协同

现代高性能程序往往组合使用多种并行技术:
示例: 图像模糊处理 (高斯滤波)

1. 任务级并行（TLP）

将图像划分为多个区域，使用多线程并行处理不同区域的像素（使用 OpenMP 或 `std::thread`）。

2. 数据级并行（DLP）

在每个区域内，对像素进行批量计算：一条 SIMD 指令同时处理 4/8/16 个像素。

3. 结果

- 多核 → 利用 TLP 加速整体任务
- SIMD → 在每个核内进一步加速局部计算
- 总加速比 $\approx$ （核心数） $\times$ （SIMD 宽度）

3 常见 SIMD 指令集架构

架构	SIMD 指令集	位宽	主要应用
x86/x86-64	MMX（已淘汰） SSE（Streaming SIMD Extensions） AVX（Advanced Vector Extensions） AVX2 AVX-512	64, 128, 256, 512 位	高性能计算 图像处理 科学计算
ARM	NEON（ARMv7） SVE（Scalable Vector Extension, ARMv8.2+） SVE2	64-128 位（NEON） 可变（128-2048 位，SVE）	移动设备 AI 推理 嵌入式系统
RISC-V	V 扩展（Vector Extension）	可配置（如 128/256/512 位）	开源架构

4 SIMD 和 GPU 并行技术的关系

4.1 SIMD 是 GPU 并行架构的理论基础

- SIMD 的核心思想：一条指令同时作用于多个数据元素。
- GPU 的设计目标：高效处理大量规则、同构的数据（如像素、顶点、神经网络张量），与 SIMD 相契合。

- 所以 GPU 在硬件层面广泛采用了 SIMD 及其演化形式来实现数据级并行。

4.2 GPU 采用 SIMT

GPU 并没有直接使用传统的 SIMD 架构，而是采用了 SIMT（Single Instruction, Multiple Threads，单指令多线程）架构。

特性	传统 SIMD	GPU 的 SIMT
执行单元	一个向量寄存器 + 向量 ALU	多个独立线程（warp/wavefront）
控制流	所有数据必须执行相同操作（无分支）	表面上是多线程，但硬件以“锁步”方式执行相同指令
分支处理	难以处理条件分支（需掩码或拆分）	支持分支，但发散分支导致性能下降
编程模型	显式向量化（如 intrinsic）	隐式并行（写标量代码，编译器/硬件自动并行）

4.3 GPU 中的 SIMD/SIMT 应用

以 NVIDIA GPU 为例：

- SM（Streaming Multiprocessor）是 GPU 的基本计算单元。
- 每个 SM 包含多个 CUDA 核心（实际的标量 ALU），这些核心共享指令调度和控制逻辑。
- 指令以 warp（32 线程）为单位调度。
- 一条加法指令在 warp 中广播到 32 个 CUDA 核心，各自处理自己的数据。

本质上是“单指令（广播）→ 多数据（每个线程一份）”，符合 SIMD 的理念。

5 SIMD 开源项目

5.1 SIMD Everywhere (SIMDe)

Github: <https://github.com/simd-everywhere/simde>

特点:

- 零依赖、纯头文件库
- 将 Intel SSE/AVX、ARM NEON、WebAssembly SIMD 等统一接口
- 自动 fallback 到标量代码（无 SIMD 时仍可运行）
- 被 FFmpeg、OpenCV、Numpy 等项目采用

适用场景: 跨平台 SIMD 移植、避免平台差异

5.2 xsimd

Github: <https://github.com/xtensor-stack/xsimd>

特点:

- C++14 模板库, 支持 SSE/AVX/AVX2/AVX-512、ARM NEON 等
- 与 xtensor (类似 Numpy 的 C++ 库) 集成
- 自动向量化数学函数 (sin, exp, log 等)
- 支持自定义类型

适用场景: 科学计算、数值模拟、机器学习底层

5.3 Highway

Github: <https://github.com/google/highway>

特点:

- Google 开发, 用于 JPEG XL、VP9/AV1 编解码器
- 支持 SSE/AVX/AVX2/AVX-512、ARM NEON 等
- 高性能、低开销、支持动态调整 (运行时选择最佳指令集)

适用场景: 图像/视频编解码、多媒体处理、高性能库